



Clínica Sonrisa Feliz

Sistema de Gestión Odontológica

INFORME TÉCNICO FINAL DE DESARROLLO

INTEGRANTES DEL GRUPO

Aranda, Fabián

Zorro, Brayan

Pabón, Gabriel

Soto, Christian

1. Introducción y descripción del sistema	3
2. Arquitectura y diseño en capas	3
2.1 Diagrama de arquitectura	4
2.2 Jerarquía de clases del modelo	4
3. Principios de POO aplicados	5
4. Funcionalidades implementadas por entrega	6
5. Ciclo de vida de un turno	7
6. Manejo de excepciones	7
7. Persistencia de datos	8
8. Interfaz gráfica — Entrega 4	8
9. Desafíos y decisiones de diseño	9
10. Conclusiones	9

1. Introducción y descripción del sistema

El presente informe documenta el desarrollo del **Sistema de Gestión Clínica Sonrisa Feliz**, una aplicación de escritorio diseñada y construida a lo largo de cuatro entregas progresivas en el marco de la materia *Paradigma Orientado a Objetos* de la Universidad Argentina de la Empresa (UADE).

El sistema permite a los recepcionistas de una clínica odontológica gestionar pacientes, odontólogos y turnos de manera íntegra: desde el alta y la baja hasta la búsqueda avanzada, pasando por el control del ciclo de vida de cada turno (PENDIENTE → CONFIRMADO → COMPLETADO / CANCELADO). Los datos se persisten automáticamente en archivos CSV y la interfaz, desarrollada en Java Swing, ofrece una experiencia de usuario amigable con validaciones visuales en tiempo real.

El sistema fue implementado íntegramente en **Java SE 23**, sin dependencias externas, aplicando los pilares del paradigma orientado a objetos y los patrones de diseño estudiados durante la cursada.

2. Arquitectura y diseño en capas

El proyecto adopta una arquitectura de **cuatro capas** claramente separadas, inspirada en el patrón *Model-View-Controller* (MVC) y extendida con una capa de repositorios para el acceso a datos. Esta separación garantiza bajo acoplamiento, alta cohesión y facilidad de extensión o reemplazo de cada capa de forma independiente.

2.1 Diagrama de arquitectura

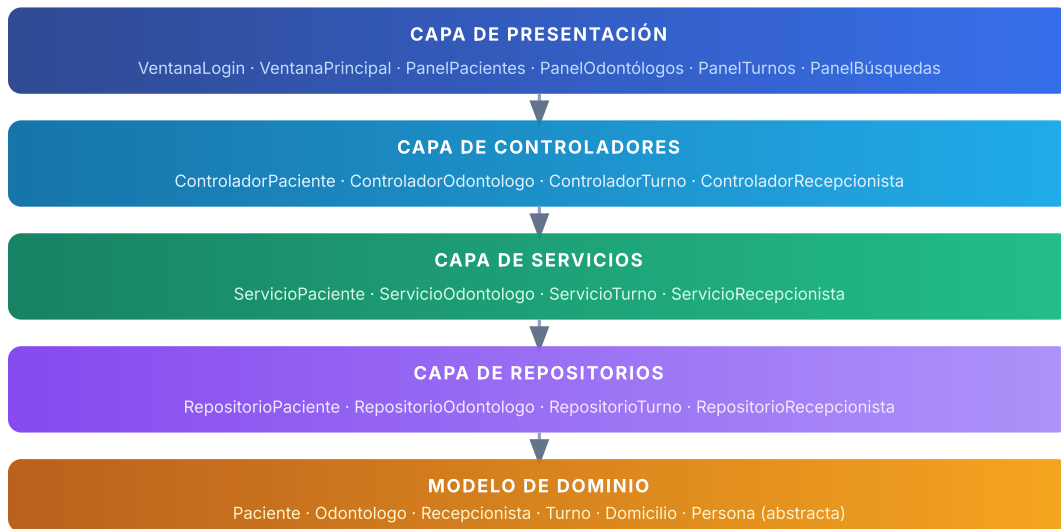


Figura 1. Diagrama de arquitectura en capas del sistema. Las dependencias fluyen siempre de arriba hacia abajo; las capas superiores nunca conocen los detalles de implementación de las inferiores.

2.2 Jerarquía de clases del modelo

El modelo de dominio se construye alrededor de una clase abstracta `Persona` que centraliza los atributos comunes (nombre, apellido, email) y delega las particularidades a cada subclase concreta mediante herencia.



Figura 2. Jerarquía de herencia del modelo de dominio. `Persona` es una clase abstracta; las subclases concretas extienden su comportamiento con atributos propios de cada rol dentro de la clínica.

3. Principios de POO aplicados

Principio	Aplicación en el sistema
Abstracción	La clase abstracta <code>Persona</code> define el contrato común de todos los actores del sistema (nombre, apellido, email, <code>getNombreCompleto()</code>) sin instanciarse directamente.
Encapsulamiento	Todos los atributos son <code>private</code> o <code>final</code> . El acceso se realiza exclusivamente a través de getters/setters. Las validaciones de negocio residen en los Servicios y Controladores, nunca expuestas al exterior.
Herencia	<code>Paciente</code> , <code>Odontologo</code> y <code>Recepcionista</code> heredan de <code>Persona</code> . La jerarquía de excepciones parte de <code>ClinicaException</code> (que extiende <code>RuntimeException</code>), con 9 subclases concretas.
Polimorfismo	Los controladores reciben y retornan instancias del tipo abstracto <code>Persona</code> cuando es posible. Los <code>AbstractTableModel</code> de Swing polimórficamente sirven a cualquier <code>JTable</code> .
Interfaces / Comparable	La clase <code>Turno</code> implementa <code>Comparable<Turno></code> , lo que permite ordenar colecciones de turnos cronológicamente sin lógica externa.
Stream API	Los repositorios usan <code>stream().filter().collect()</code> para todas las búsquedas y filtrados, reemplazando bucles <code>for</code> imperativo por código declarativo y funcional.
Colecciones avanzadas	Se emplean <code>TreeMap</code> , <code>TreeSet</code> y <code>LinkedHashMap</code> según la necesidad de orden y acceso. <code>Agenda</code> usa un <code>TreeMap<LocalDate, TreeSet<LocalTime>></code> para detectar conflictos de horario en $O(\log n)$.

4. Funcionalidades implementadas por entrega

Entrega 1 — Modelo de Dominio y Fundamentos POO

Se construyó el modelo de dominio completo del sistema. La clase abstracta `Persona` (con métodos abstractos `getRol()` e `getIdentificador()`) sirvió de base para las tres entidades concretas: `Paciente` (con DNI y composición con `Domicilio`), `Odontologo` (con matrícula y composición con `Agenda`) y `Recepcionista` (con legajo). La clase `Turno` implementó `Comparable<Turno>` para ordenamiento cronológico y el método `esFuturo()`. La clase `Agenda` empleó un `TreeMap<LocalDate, TreeSet<LocalTime>>` para gestionar horarios con detección de conflictos en $O(\log n)$. Se definió el enum `EstadoTurno` con los cuatro estados del ciclo de vida. Todos los IDs se autogeneraron mediante contadores estáticos.

Conceptos POO: abstracción, herencia, encapsulamiento, composición, enums, interfaces (`Comparable`), colecciones avanzadas, Java Time API.

Entrega 2 — Arquitectura en Capas y Lógica de Negocio

Se implementó la arquitectura completa de cuatro capas. En la capa de repositorios, se definió la **interfaz genérica** `IRepositorio<T>` con operaciones CRUD, implementada por `RepositorioPaciente`, `RepositorioOdontologo`, `RepositorioTurno` y `RepositorioRecepcionista`, todos usando **Stream API** (`stream().filter().collect()`) para las búsquedas. Los Servicios encapsulan la lógica de negocio: validación de DNI/matriculación duplicados, gestión del ciclo de vida de turnos (transiciones de estado con validación), e integridad referencial manual (verificación de turnos futuros antes de permitir una baja). Los Controladores centralizan las validaciones de formato de entrada antes de delegar a los Servicios. Las dependencias se inyectan por constructor en todos los niveles.

Conceptos POO: interfaces genéricas, polimorfismo, Stream API, inyección de dependencias básica, principio de responsabilidad única (SRP).

Entrega 3 — Excepciones Personalizadas y Persistencia CSV

Se diseñó una jerarquía de **9 excepciones personalizadas** bajo `ClinicaException` (que extiende `RuntimeException`): `DatoInvalidoException`, `DniDuplicadoException`, `MatriculaDuplicadaException`, `TurnoYaReservadoException`, `FechaInvalidaException`, `PacienteNoEncontradoException`, `OdontologoNoEncontradoException`, `TurnoNoEncontradoException` y `OperacionNoPermitidaException`. La persistencia se implementó con la clase `PersistenciaCSV` usando bloques `try-catch-finally` para garantizar el cierre de recursos y **Shutdown Hooks** (`Runtime.getRuntime().addShutdownHook()`) para asegurar el guardado automático al cerrar la JVM. Los repositorios fueron refactorizados para usar Stream API en todas las búsquedas.

Conceptos POO: jerarquía de excepciones, manejo de errores, persistencia con I/O de archivos, shutdown hooks.

Entrega 4 — Interfaz Gráfica con Java Swing

Se reemplazó por completo la interfaz de consola por una GUI profesional construida en Java Swing estándar. `VentanaLogin` maneja la autenticación; `VentanaPrincipal` contiene el sidebar de navegación con `CardLayout`. Los paneles de gestión (`PanelPacientes`, `PanelOdontologos`) implementan `AbstractTableModel` personalizado para las `JTable`, con `MouseListener` para autocompletar formularios. `PanelTurnos` usa `JComboBox` dinámicos y `SwingWorker` para carga asíncrona (no bloqueante) de la lista de turnos. La validación visual marca los campos erróneos con borde rojo antes de enviar datos al controlador. El cierre de la ventana es interceptado por un `WindowListener` que solicita confirmación.

Conceptos POO: `AbstractTableModel`, listeners de eventos (Action, Mouse, Window), `SwingWorker` (hilos), `CardLayout`, patrones Observer implícitos de Swing.

5. Ciclo de vida de un turno

Un turno atraviesa un ciclo de vida definido por el enum `EstadoTurno`. Las transiciones entre estados son validadas por `ServicioTurno`, que lanza una `OperacionNoPermitidaException` si se intenta una transición inválida.



Figura 3. Diagrama de máquina de estados del ciclo de vida de un `Turno`. Los estados **COMPLETADO** y **CANCELADO** son finales; ninguna transición los abandona.

6. Manejo de excepciones

Se diseñó una jerarquía de excepciones propia que extiende `RuntimeException` a través de la clase base `ClinicaException`. Esto permite que las capas superiores (Controladores, GUI) capturen excepciones de negocio de forma polimórfica, sin necesidad de declarar `throws` en cada método.

Excepción	Cuándo se lanza
<code>DatoInvalidoException</code>	Campo nulo, vacío o con formato incorrecto en validaciones de los Controladores.
<code>DniDuplicadoException</code>	Se intenta registrar un paciente con un DNI ya existente en el repositorio.
<code>MatriculaDuplicadaException</code>	Se intenta registrar un odontólogo con una matrícula ya existente.
<code>TurnoYaReservadoException</code>	El odontólogo ya tiene un turno en la fecha y hora exactas solicitadas.
<code>FechaInvalidaException</code>	Se intenta reservar un turno en una fecha pasada.
<code>PacienteNoEncontradoException</code>	Búsqueda por ID o DNI que no encuentra ningún paciente.
<code>OdontologoNoEncontradoException</code>	Búsqueda que no encuentra al odontólogo solicitado.
<code>TurnoNoEncontradoException</code>	Operación sobre un turno con ID inexistente.
<code>OperacionNoPermitidaException</code>	Transición de estado inválida o eliminación de entidad con dependencias activas.

i La GUI captura todas las excepciones de tipo `ClinicaException` y muestra su mensaje al usuario mediante `JOptionPane`, sin exponer el stack trace. Las excepciones inesperadas se muestran con un mensaje genérico.

7. Persistencia de datos

La persistencia se implementa mediante la clase `PersistenciaCSV`, que serializa y deserializa las colecciones de entidades a archivos de texto con valores separados por comas (`.csv`). No se utilizan frameworks externos ni bases de datos.

Mecanismo de guardado automático

Al iniciarse la aplicación, se registra un **Shutdown Hook** mediante `Runtime.getRuntime().addShutdownHook()`. Este hilo se ejecuta automáticamente cuando la JVM termina (ya sea por cierre normal de la ventana, uso del botón Salir, o interrupción del proceso), garantizando que los datos siempre sean persistidos independientemente de cómo finalice la aplicación.

Adicionalmente, cada operación de alta, baja o modificación invoca inmediatamente el método de guardado del repositorio correspondiente, lo que elimina el riesgo de pérdida de datos en caso de cierre inesperado.

Archivos generados

Los archivos se almacenan en la carpeta `datos/` relativa al directorio de ejecución del JAR:



datos/pacientes.csv



datos/odontologos.csv



datos/turnos.csv



datos/recepcionistas.csv

8. Interfaz gráfica

La interfaz gráfica fue construida exclusivamente con las bibliotecas estándar de Java Swing, sin frameworks de UI adicionales. La arquitectura de la presentación separa los componentes de visualización de la lógica de negocio, respetando la arquitectura en capas del sistema.

Componentes principales

Clase	Tipo Swing	Responsabilidad
VentanaLogin	JDialog	Autenticación y registro del recepcionista antes de ingresar al sistema.
VentanaPrincipal	JFrame	Contenedor principal con sidebar de navegación y <code>CardLayout</code> para los paneles.
PanelPacientes	JPanel	CRUD de pacientes con <code>JTable</code> , formulario y validación visual.
PanelOdontologos	JPanel	CRUD de odontólogos, análogo al panel de pacientes.
PanelTurnos	JPanel	Gestión completa de turnos con <code>JComboBox</code> dinámicos y <code>SwingWorker</code> .
PanelBusquedas	JPanel	Búsquedas avanzadas con 7 criterios y tabla de resultados dinámica.
ModeloTabla*	AbstractTableModel	Tres implementaciones para Pacientes, Odontólogos y Turnos respectivamente.

Eventos implementados

- `ActionListener`: en todos los botones de acción (guardar, eliminar, buscar, reservar, confirmar, cancelar, etc.).
- `MouseListener`: en las tres `JTable` para autocompletar el formulario al hacer clic en una fila.
- `WindowListener`: en `VentanaPrincipal` para interceptar el cierre de ventana y solicitar confirmación.
- `SwingWorker`: en `PanelTurnos` para cargar la lista de turnos en un hilo secundario (`doInBackground()`), evitando el congelamiento del Event Dispatch Thread durante operaciones de I/O.

9. Desafíos y decisiones de diseño

Gestión de conflictos de horario

Para detectar turnos superpuestos sin recorrer toda la lista de turnos, se implementó la clase `Agenda` dentro de `Odontologo`. Internamente, utiliza un `TreeMap<LocalDate, TreeSet<LocalTime>>`: la clave primaria es la fecha y el valor es el conjunto ordenado de horarios reservados en esa fecha. La detección de conflicto tiene costo logarítmico $O(\log n)$, más eficiente que una búsqueda lineal.

Sincronización entre capas al navegar la GUI

Al navegar al panel de turnos, el sistema llama a `actualizarCombos()`, que recarga los `JComboBox` de pacientes y odontólogos con los datos más recientes del repositorio. Esto garantiza que los combos reflejen altas o bajas realizadas durante la misma sesión sin necesidad de reiniciar la aplicación.

Integridad referencial sin base de datos

Sin una base de datos relacional que aplique restricciones de clave foránea, la integridad fue implementada manualmente: antes de eliminar un paciente u odontólogo, el servicio consulta el repositorio de turnos para verificar que no existan turnos futuros asociados, lanzando `OperacionNoPermitidaException` si los hay.

Reconstrucción del estado del contador de IDs desde CSV

Dado que los IDs de las entidades se generan de forma autoincremental mediante un campo estático (`contadorId`), al cargar datos desde CSV se calibra el contador para que siempre sea mayor al ID más alto existente, evitando colisiones de identidad entre sesiones.

10. Conclusiones

El desarrollo del Sistema de Gestión Clínica Sonrisa Feliz a lo largo de las cuatro entregas permitió al equipo aplicar de manera progresiva y práctica los conceptos centrales del paradigma orientado a objetos: diseño de clases con herencia y polimorfismo, encapsulamiento de la lógica de negocio, separación de responsabilidades mediante capas, manejo robusto de errores y construcción de interfaces de usuario funcionales y amigables.

La arquitectura en capas adoptada demostró ser altamente mantenible: la migración de la interfaz de consola a Swing en la Entrega 4 no requirió modificar ninguna línea de código en las capas de Servicio, Repositorio o Modelo, lo que valida la correcta separación de responsabilidades lograda desde las primeras entregas.

El uso de colecciones avanzadas (`TreeMap` , `TreeSet`), la Stream API, los Shutdown Hooks para persistencia y el `SwingWorker` para operaciones asíncronas en la GUI son evidencia de la aplicación de buenas prácticas de programación en Java más allá de los requisitos mínimos de la consigna.

- ✓ El sistema compilado se distribuye como un archivo ejecutable `ClinicaSonrisaFeliz.jar` que puede ejecutarse en cualquier equipo con Java 17+ instalado, sin configuración adicional.